Yandex Cloud

Облачные технологии в работе с большими данными. День 6

Баташов Руслан Ансарович,
Customer Success Manager

Автоматизация

1. Работа с CLI. (Перезагрузка ВМ, создание ВМ, использование файлов спецификаций)
2. Packer
3. Terraform. Создание ВМ.
4. DevOps. Микросервисы. Контейнеры
5. Создание docker-образа и загрузка в Container Registry.
6. Kubernetes. Создание кластера, размещение приложения, балансировка нагрузки, масштабирование.

Kubernetes

Открытое программное обеспечение для оркестрации* контейнеризированных приложений

Оркестрация - отслеживание работоспособности и перезапуск при сбоях, обновление, разворачивание, масштабирование, остановка*

Что делает Kubernetes

Автоматическое развёртывание

Вы можете описать состояние контейнеров в виде конфигурации, и Kubernetes автоматически обеспечит заданное состояние: будет развёртывать и удалять контейнеры, перераспределять ресурсы.

Мониторинг сервисов и балансировка.

Kubernetes распределяет сетевой трафик так, чтобы развёртывание было стабильным.

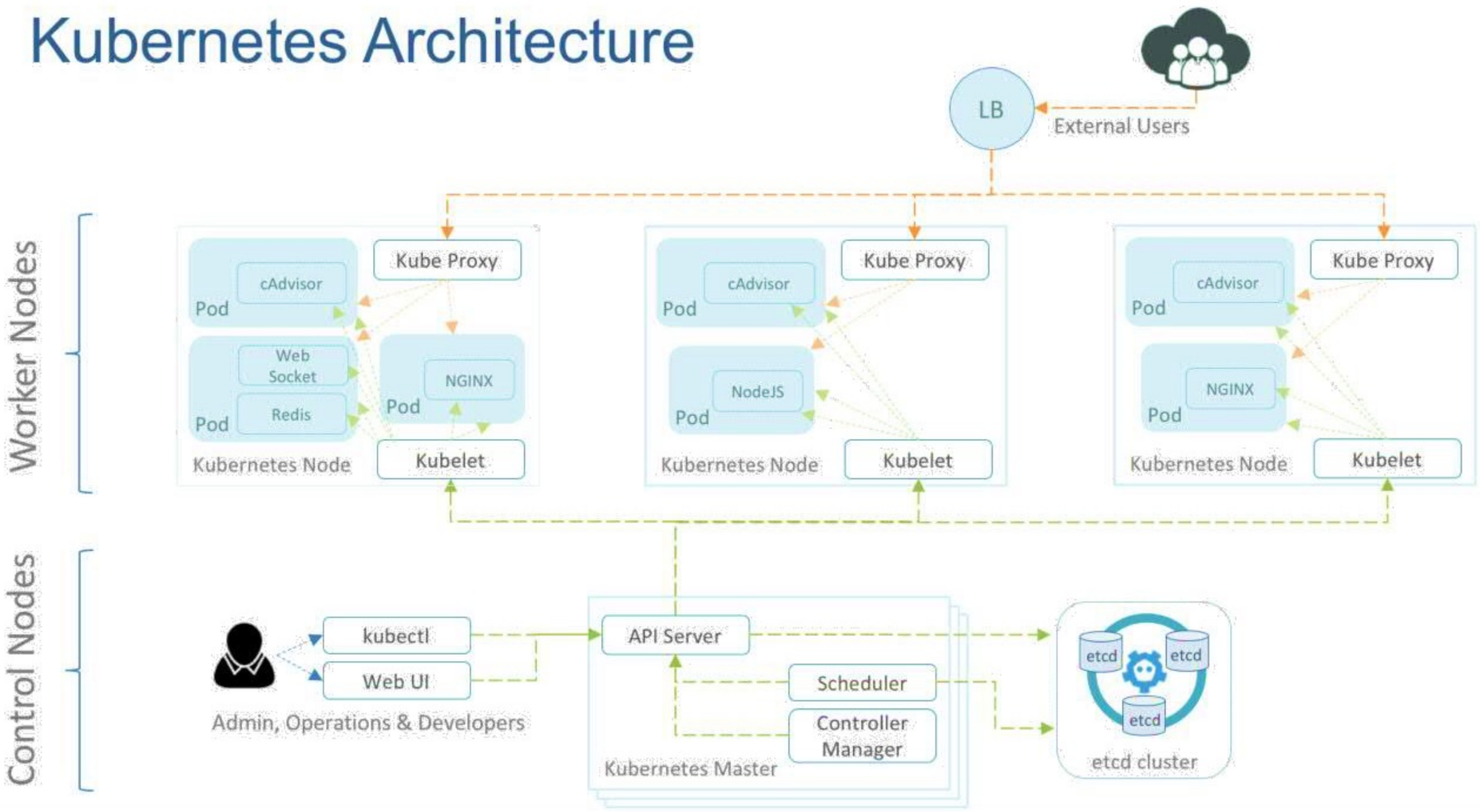
Оркестрация хранилища

Kubernetes позволяет автоматически смонтировать систему хранения: локальное или облачное хранилище.

Самоконтроль

Kubernetes перезапускает отказавшие контейнеры, заменяет их и завершает работу контейнеров, которые не соответствуют заданному уровню работоспособности

K8s схема


Kubernetes Architecture
LB
External Users
Worker Nodes
Pod
cAdvisor
Kube Proxy
Web Socket
Redis
NGINX
Kubelet
Kubernetes Node
NodeJS
Control Nodes
kubectl
Web UI
Admin, Operations & Developers
API Server
Scheduler
Controller Manager
Kubernetes Master
etcd
etcd cluster

K8s.Основные понятия

Master управляет worker'ами, распределяет по ним контейнеры, управляет рабочими нагрузками, следит за тем чтобы приложения и кластеры были в соответствующем состоянии.

Etcd - хранилище, которое в кластере kubernetes используется для хранения конфигураций. В хранилище лежит информация о всех компонентах и их состояниях в кластере на текущий момент (реально и "как должно быть").

API server - Точка управления кластером - через api server пользователи конфигурируют "рабочую нагрузку": deployments, roles, workloads.

Scheduler - Распределяет всю рабочую нагрузку по нодам кластера.

Proxy - Нужен, чтобы сетевая инфраструктура была доступной и предсказуемой, в то же время - изолированной.

Nodes - машина в кластере Kubernetes.

Pods - группа контейнеров с общими разделами, запускаемых как единое целое.

Kubectl - интерфейс командной строки для управления Kubernetes.

Kubelet – агент, запущенный на нодах, который управляет pod'ами их контейнерами, образами, разделами, etc.

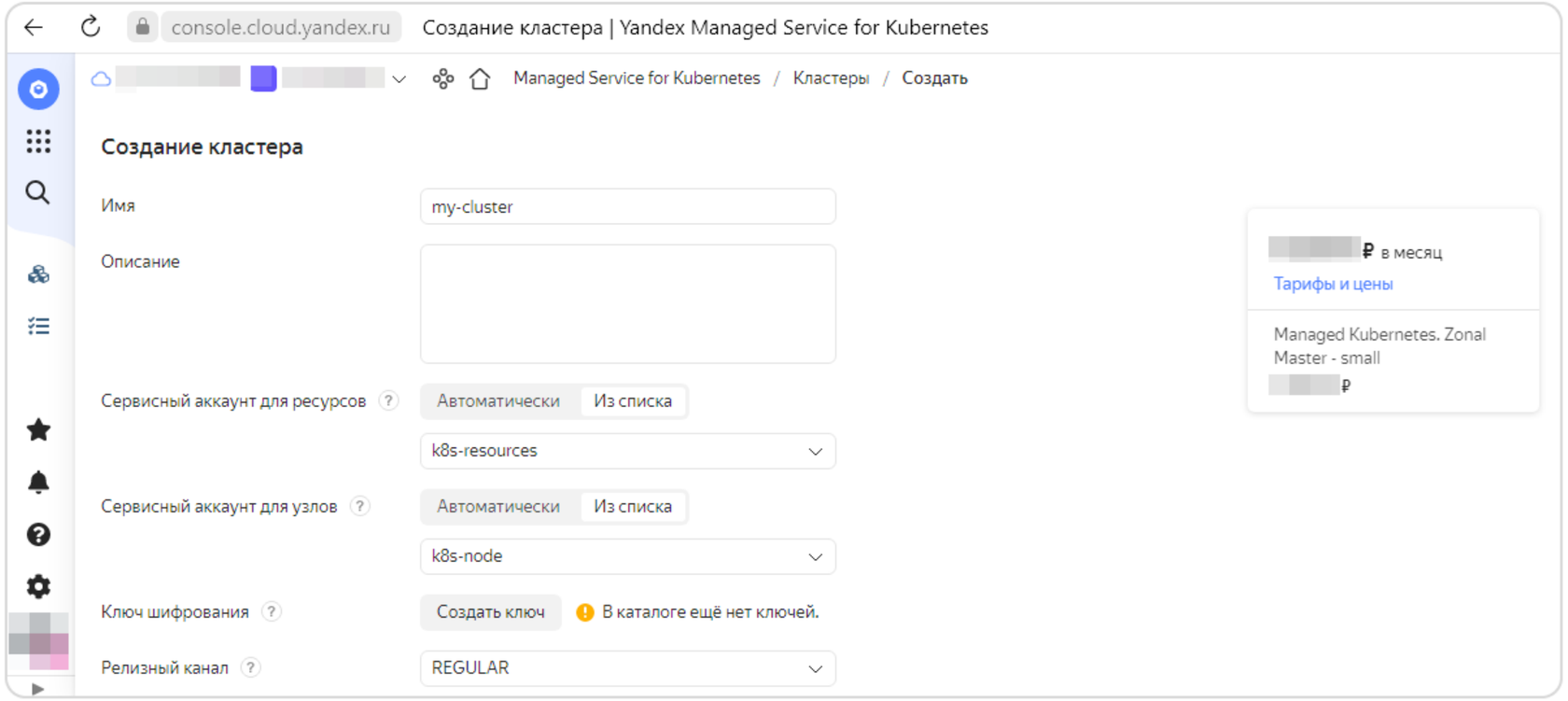
Managed K8s

Процессы	Managed Service for Kubernetes	Самостоятельная инсталляция Kubernetes
Развёртывание виртуальных машин	●	○
Настройка сети	●	○
Установка ОС и ПО	●	○
Обновление кластера и групп узлов	●	○
Доступность мастера	●	○
Резервное копирование	●	○
Безопасность хранилища данных и оборудования	●	○
Интеграция с сервисами Yandex Cloud	●	○

○ Самостоятельный контроль ● Контроль на стороне Yandex Cloud

В этой практической работе вы создадите кластер Kubernetes и группу узлов в нём.

1. Выберите сервис **Managed Service for Kubernetes**. Нажмите кнопку **Создать кластер**. Дальше заполним настройки кластера:



2. Для Kubernetes необходим сервисный аккаунт для ресурсов и узлов.

Сервисный аккаунт для ресурсов — это аккаунт, под которым сервису Kubernetes будут выделяться ресурсы в нашем облаке.

Сервисный аккаунт для узлов необходим уже созданным узлам самого кластера Kubernetes для доступа к другим ресурсам. Например, чтобы получить Docker-образы из Container Registry.

Этим аккаунтам нужны разные права, и поэтому у них бывают разные роли. В общем случае вы можете использовать один и тот же сервисный аккаунт. Выберите аккаунт, который создали на первом курсе, или заведите новый.

3. Ключ шифрования Yandex Key Management Service позволяет защитить конфиденциальную информацию (пароли, OAuth-токены и SSH-ключи) и повысить безопасность. Это необязательно — кластер запустится и без ключа. Для этой практической работы не создавайте его.

4. Релизные каналы RAPID, REGULAR и STABLE отличаются процессом обновления и доступными вам версиями Kubernetes.

RAPID и REGULAR содержат все версии, включая минорные. STABLE — только стабильные версии. RAPID обновляется автоматически, а в REGULAR и STABLE обновление можно отключить. Когда появляется обновление, информация о нём отображается в консоли управления.

Выберите REGULAR.

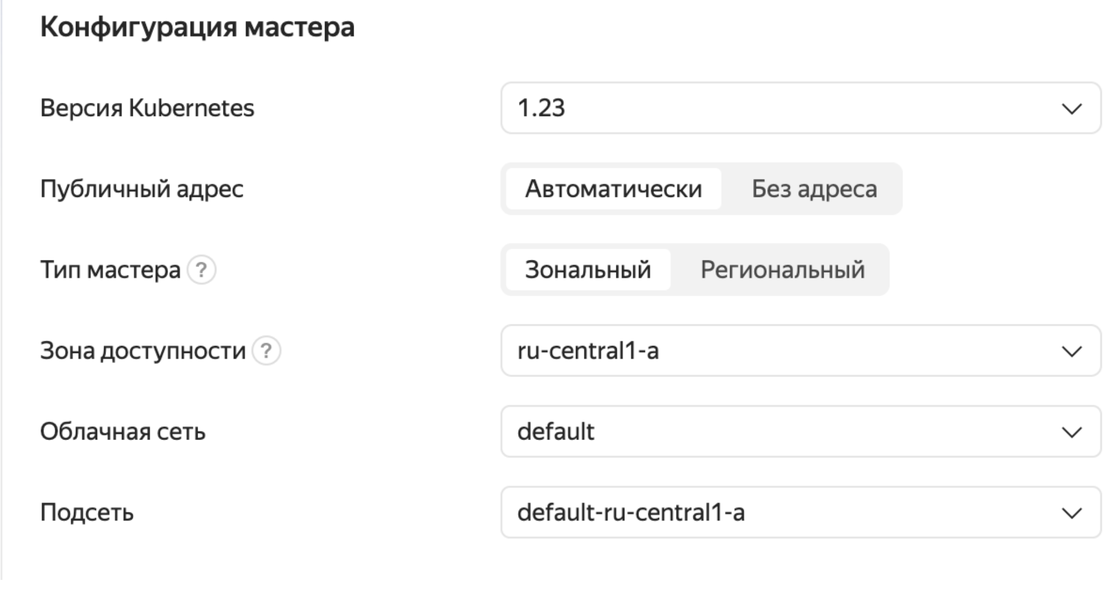
Конфигурация мастера

5. Выберите версию Kubernetes. Их набор зависит от релизного канала. Версии мастера и других нод могут не совпадать, но это достаточно тонкая настройка, могут возникнуть проблемы совместимости, которые повлияют на работу всего кластера.

6. Кластеру может назначаться публичный IP-адрес. Выберите вариант **Автоматически**. В этом случае IP выбирается из пула свободных IP-адресов.

7. Тип мастера влияет на отказоустойчивость. **Зональный** работает только в одной зоне доступности, а **региональный** — в трёх подсетях в каждой зоне доступности.

8. Выбор типа мастера также влияет на подсети, в которых будет развёрнут кластер. У вас уже есть подсети, созданные по умолчанию для функционирования облака. Выберите их.



Настройка окна обновлений

9. Режимов обновления четыре: **Отключено**, **В любое время**, **Ежедневно** и **В выбранные дни**. Региональный мастер во время обновления остаётся доступен, зональный — нет.

   Группа узлов кластера обновляется с выделением дополнительных ресурсов, так как при обновлении создаются узлы с обновлённой конфигурацией. При обновлении поды с контейнерами будут переезжать с одного узла на другой.

   По умолчанию выставлен пункт В любое время. Оставьте его.

Сетевые настройки кластера

9. **Сетевые политики** для кластера Kubernetes необязательны. Эта опция включает сетевой контроллер Calico, который позволяет применять тонкие настройки политик доступа для кластера. Не выбирайте эту опцию.

Во время работы кластера подам с контейнерами и сервисам самого кластера Kubernetes будут автоматически присваиваться внутренние IP-адреса. Чтобы IP-адреса подов и сервисов Kubernetes не пересеклись с другими адресами в вашем облаке, задайте **CIDR** (Classless Inter-Domain Routing — бесклассовая междоменная маршрутизация). Оставьте адреса пустыми: они будут назначены автоматически.

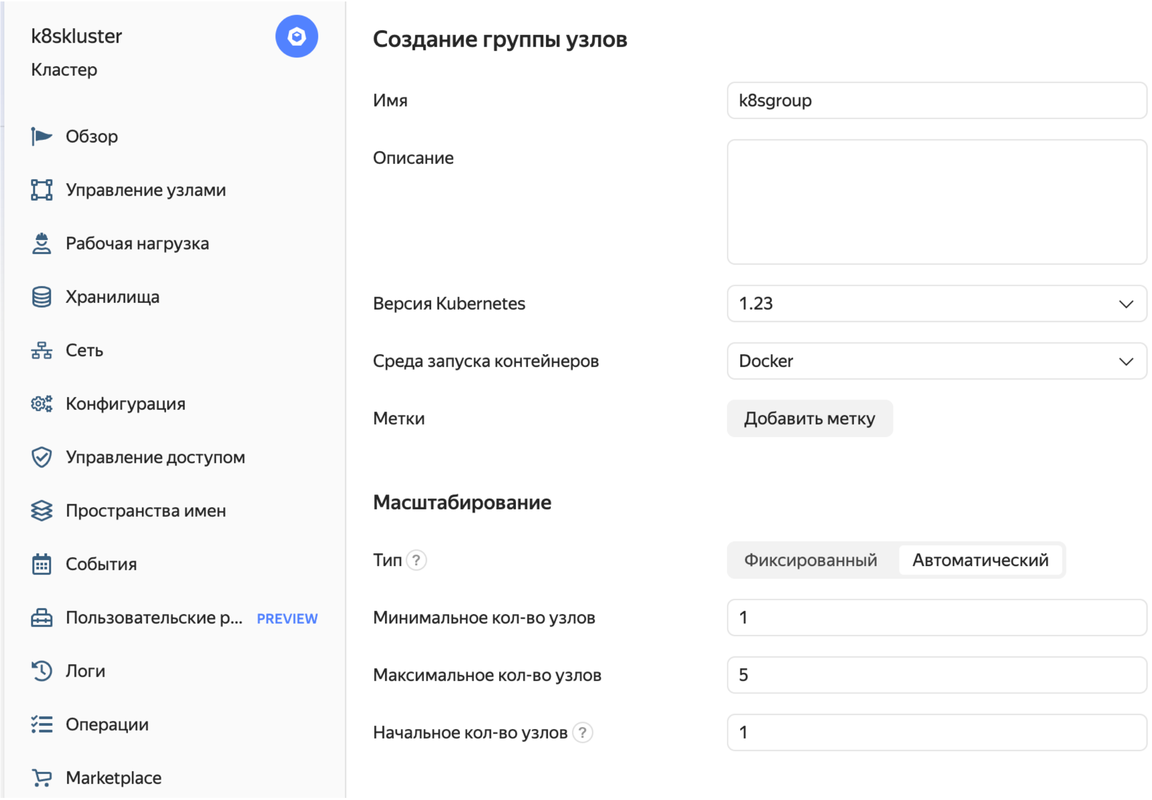
Маска подсети узлов влияет на количество подов, которые могут запускаться. Если адресов не хватит, под не запустится.

11.Вы заполнили все настройки, теперь нажмите **Создать кластер**. Дождитесь, пока статус кластера станет RUNNING, а состояние — HEALTHY. Это может занять около 10 минут.

Создание группы узлов

12. Зайдите в созданный кластер, перейдите на вкладку **Управление узлами** и нажмите **Создать группу узлов**. Группы узлов — это группы виртуальных машин.

13. Введите имя и описание группы, выберите версию Kubernetes. Выберите **Автоматический** тип масштабирования и количество узлов от 1 до 5. Укажите среду запуска контейнеров — Docker.



14. В сетевых настройках задайте автоматический IP-адрес и выберите зону доступности (кластер зональный, поэтому зона доступности только одна). Задайте SSH-ключ, чтобы иметь доступ к виртуальным машинам кластера. Настройки обновления идентичны настройкам мастера.

15. Остальные настройки группы оставьте по умолчанию.

16. Нажмите **Создать группу узлов** и дождитесь, пока операция выполнится.

В этой практической работе мы развернем с помощью командной строки в кластере приложение — веб-сервер NGINX.

1.Основное средство взаимодействия с кластером — инструмент kubectl. Установите его по инструкции.

2.В консоли управления войдите в созданный кластер Managed Service for Kubernetes и нажмите кнопку **Подключиться**. В открывшемся окне скопируйте команду для подключения:

```
yc managed-kubernetes cluster get-credentials \
  --id <идентификатор_кластера> \
  --external
```

3. Чтобы проверить правильность установки и подключения, посмотрите на конфигурацию:

```
kubectl config view
```

Ответ получится примерно таким (IP-адрес сервера и название кластера будут отличаться):

```
ipivnev@ipivnev-osx ~ %  kubectl config view

apiVersion: v1
clusters:
- cluster:
    certificate-authority-data: DATA+OMITTED
    server: https://51.250.80.123
  name: yc-managed-k8s-cat3iahvd91l03ah0it0
- cluster:
    certificate-authority-data: DATA+OMITTED
    server: https://51.250.74.50
  name: yc-managed-k8s-catb871omli74ppgpa5f
```

Создание манифеста

1. Для описания настроек приложения в кластере создадим файл ***my-nginx.yaml***. Такой файл называется **манифестом**.

apiVersion: apps/v1 - определяет, для какой версии Kubernetes написан манифест.
kind: Deployment - описывает механизм использования. Она может принимать значения Deployment, Namespace, Service, Pod, LB
metadata: - определяет метаданные приложения: имя, метки (labels), аннотации.
* name: my-nginx-deployment*
spec: - содержится описание объектов Kubernetes.
* replicas: 1* - определяет масштабирование подов
* selector:* -определяет, какими подами будет управлять контейнер. Поды отбираются с помощью метки
*   matchLabels:*
*   app: nginx*
* template:* - определяет шаблон пода. Метка в шаблоне должна совпадать с меткой селектора — nginx.
*   metadata:*
*   labels:*
*   app: nginx*
*   spec:* - Она задаёт настройки контейнеров, которые будет развёрнуты на поде. Нам нужен один контейнер. Используйте для него образ, созданный ранее с помощью Docker и помещённый в реестр Yandex Container Registry.
*   containers:*
*   - name: nginx*
*   image: cr.yandex/<идентификатор_реестра>/ubuntu-nginx:latest*

Настройки манифеста для развёртывания приложения есть в документации Kubernetes.

Выполнение манифеста

1. Для создания или обновления ресурсов в кластере используется команда **apply**. Файл манифеста указывается после флага **-f**.

   kubectl apply -f <путь_к_файлу_my-nginx.yaml>

   Если результат будет успешным, вы увидите сообщение:

   deployment.apps/my-nginx-deployment created

2. Чтобы убедиться, что приложение создано, посмотрите список подов:

   kubectl get pods

3. Теперь получите более подробную информацию:

   kubectl get pods -o wide

4. Чтобы получить максимально подробную информацию:

   kubectl describe deployment/my-nginx-deployment

Масштабирование

1. Теперь увеличьте количество подов. Вручную это можно сделать двумя способами:

   - изменить файл манифеста, указав в директиве replicas нужное число подов, и снова выполнить команду apply;
   - если файла манифеста нет под рукой — использовать команду scale:

kubectl scale --replicas=3 deployment/my-nginx-deployment

В результате *kubectl get pods* получим:

```
ipivnev@ipivnev-osx ~ % kubectl scale --replicas=3 deployment/my-nginx-deployment
deployment.apps/my-nginx-deployment scaled
ipivnev@ipivnev-osx ~ % kubectl get pods
NAME                                  READY   STATUS    RESTARTS   AGE
my-nginx-deployment-6f46bc6ff8-bwwf2   1/1     Running   0          11s
my-nginx-deployment-6f46bc6ff8-dq2jv   1/1     Running   0          11s
my-nginx-deployment-6f46bc6ff8-jpphx   1/1     Running   0          2m54s
```

Балансировка нагрузки

Вы развернули в кластере приложение, но у вас пока нет к нему доступа из интернета. Чтобы исправить эту проблему, воспользуемся сервисом LoadBalancer.
В кластере есть собственный сервис DNS. Он работает с внутренними IP-адресами объектов кластера, чтобы те могли взаимодействовать. Однако внутренний IP-адрес может меняться, когда ресурсы группы узлов обновляются. Чтобы обращаться к приложению извне, требуется неизменный публичный IP-адрес — это и будет IP-адрес балансировщика.

1. Создайте файл-манифест **load-balancer.yaml**:

```
apiVersion: v1
kind: Service
metadata:
  name: my-loadbalancer
spec:
  selector:
    app: nginx
  ports:
  - port: 80
    targetPort: 80
  type: LoadBalancer
```

Где:
- port — порт сетевого балансировщика, на котором будут обслуживаться пользовательские запросы;
- targetPort — порт контейнера, на котором доступно приложение;
- selector — метка селектора из шаблона подов в манифесте объекта Deployment.

2. Выполните манифест:

kubectl apply -f <путь_к_файлу_load-balancer.yaml>

Вы увидите сообщение:

service/my-loadbalancer created

3. В консоли управления откройте раздел **Load Balancer.** Там должен появиться балансировщик нагрузки с префиксом k8s в имени и уникальным идентификатором кластера Kubernetes.

4. Скопируйте IP-адрес балансировщика в адресную строку браузера. Вы увидите приветственную страницу NGINX.

Автомасштабирование в Managed Kubernetes

Масштабирование позволяет распределять нагрузку между контейнерами и снизить риск сбоя.Для автомасштабирования подходят инструменты, встроенные в Kubernetes: **Horizontal Pod Autoscaler** и **Cluster Autoscaler**. Они решают разные задачи и могут работать как по отдельности, так и совместно. Horizontal Pod Autoscaler, как понятно из названия, масштабирует поды: увеличивает и уменьшает их количество, когда изменяется нагрузка. Cluster Autoscaler управляет количеством узлов, на которых поды запущены.

Horizontal Pod Autoscaler

Horizontal Pod Autoscaler анализирует нагрузку на сервис и исходя из неё создаёт или удаляет поды. Сервис ориентируется на лимиты (limits) и запросы (requests). Первые ограничивают ресурсы, доступные поду с контейнерами: процессор, память и др. Если их не указать, контейнер может забрать все ресурсы ноды. Запросы описывают количество свободных ресурсов, которыми должен располагать узел, чтобы на нём можно было запустить ещё один под с сервисом. Если ресурсов недостаточно, придётся создать дополнительный узел.

Cluster Autoscaler

Cluster Autoscaler оценивает запросы подов и автоматически изменяет количество узлов кластера Kubernetes:

- Если из-за нехватки ресурсов не удаётся запустить поды, то новые узлы создаются и добавляются в кластер.
- Если узлы недостаточно утилизируются, а их поды можно перенести на другие узлы, то узлы освобождаются и удаляются из кластера.

В Yandex Managed Service for Kubernetes инструмент Cluster Autoscaler включён по умолчанию. Читайте об этом в разделе документации об автомасштабировании группы узлов

1. Создайте файл-манифест **load-balancer-hpa.yaml**:

   Для начала скопируйте в него настройки спецификаций, которые вы составляли ранее: из **my-nginx.yaml** (в примере ниже это раздел Deployment) и из **load-balancer.yaml** (раздел Service).Поскольку новый балансировщик должен отслеживать отдельную группу контейнеров, используйте для контейнеров другие метки (labels), например **nginx-hpa**.

```yaml
---
### Deployment
apiVersion: apps/v1
kind: Deployment
metadata:
  name: my-loadbalancer-hpa
  labels:
    app: nginx-hpa
spec:
  replicas: 1
  selector:
    matchLabels:
        app: nginx-hpa
  template:
    metadata:
        name: nginx-hpa
        labels:
          app: nginx-hpa
spec:
        containers:
          - name: nginx-hpa
            image: k8s.gcr.io/hpa-example

---
### Service
apiVersion: v1
kind: Service
metadata:
  name: my-loadbalancer-hpa
spec:
  selector:
    app: nginx-hpa
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
  type: LoadBalancer
```

2. В разделе Deployment смените образ с Yandex Container Registry на k8s.gcr.io/hpa-example — это специальный тестовый образ из публичного репозитория, создающий высокую нагрузку на процессор. Так вам будет удобно отслеживать работу Horizontal Pod Autoscaler.

```
...
    spec:
      containers:
        - name: nginx-hpa
          image: k8s.gcr.io/hpa-example
```

3. Теперь добавьте в шаблон контейнера настройки requests и limits: мы попросим по умолчанию 256 мебибайтов памяти и 500 милли-CPU (половину ядра), а ограничим контейнер 500 мебибайтами и 1 CPU.

```
...
  spec:
   containers:
    - name: nginx-hpa
      image: k8s.gcr.io/hpa-example
      resources:
        requests:
          memory: "256Mi"
          cpu: "500m"
        limits:
          memory: "500Mi"
          cpu: "1"
```

4. Дополните манифест настройками для Horizontal Pod Autoscaler:

```
apiVersion: autoscaling/v1
kind: HorizontalPodAutoscaler
metadata:
  name: my-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: my-nginx-deployment-hpa
  minReplicas: 1
  maxReplicas: 5
  targetCPUUtilizationPercentage: 20
```

5. Примените манифест:

```
kubectl apply -f <путь_к_load-balancer-hpa.yaml>
```

Вы увидите три сообщения:

deployment.apps/my-nginx-deployment-hpa created
service/my-loadbalancer-hpa created
horizontalpodautoscaler.autoscaling/my-hpa created

7. Запустите в отдельном окне отслеживание интересующих вас компонентов кластера Kubernetes:

MAC OS/Linux: while true; do kubectl get pod,svc,hpa,nodes -o wide; sleep 5; done

Windows: for /l %q in (0) do kubectl get pod,svc,hpa,nodes -o wide

8. Теперь сымитируйте рабочую нагрузку на приложение. Для этого подойдёт утилита wget (установите её с помощью пакетного менеджера или с сайта).
   Windows:
   - Скачать ZIP файл отсюда: https://eternallybored.org/misc/wget/
   - Распаковать в папку на диске C.
   - Прописать в командной строке: set PATH=%PATH%;\путь к папке, в которой лежит wget.exe

Либо создайте новую ВМ на Ubuntu, подключитесь к ней по SSH. Проверьте, что утилита работает:

wget –version

Запустите нагрузку с помощью wget:

MAC OS/Linux: while true; do wget -q -O- <IP - адрес балансировщика>; done

Windows: for /l %q in (0) do wget -q -O- <IP - адрес балансировщика>;

Вы увидите, что сначала увеличится число подов, а затем добавятся узлы. Число узлов ограничено настройками группы узлов кластера, которые вы задали при создании кластера (в нашем случае максимальное количество узлов — пять).

9. Остановите цикл создания нагрузки на приложение (комбинация клавиш Ctrl + C). В окне консоли с отслеживанием компонентов кластера вы увидите, как удаляются узлы и поды без нагрузки.

Мониторинг в Managed Kubernetes

1. В веб-консоли перейдите в раздел **Managed Service for Kubernetes**, войдите в свой кластер и переключитесь на панели слева на вкладку **Рабочая нагрузка**. Перейдите на вкладку **Контроллеры Deployment**.
Вы увидите список запущенных сервисов. В нём будет ваш сервис my-loadbalancer-hpa. Войдите в него. На вкладках вы можете посмотреть количество и статус подов, события и другие данные.

2. На вкладке Поды вы увидите количество и статус подов, которые поддерживают сервис. Некоторые поды не созданы — у них в колонке Узел стоят прочерки.

3. Выберите вкладку **Управление узлами** , выберите Узлы и перейдите на вкладку **Мониторинг**

Здесь можно наблюдать нагрузку на CPU, которую мы запустили ранее.